

ESTRATÉGIA & ALGORITMOS

Otimização de Custos em Byteland

T290-16

Aaron Magno, Arthur Soares e Davi de Paula

Contexto do Problema

A necessidade de eficiência energética e redução de gastos.

| O Desafio Econômico

Economia em Tempos Difíceis

Byteland enfrenta restrições orçamentárias. Atualmente, cada metro de estrada iluminada custa 1 dólar por dia.

O governo decidiu desativar a iluminação de certas estradas para economizar, mantendo a conectividade mínima para garantir a segurança dos habitantes.

Requisitos do Sistema



Conectividade

Deve existir pelo menos um caminho iluminado entre quaisquer duas junções em Byteland.



Minimização

O custo diário de iluminação deve ser reduzido ao valor mínimo absoluto possível.



Segurança

A otimização não pode deixar áreas isoladas, preservando a percepção de segurança.

Modelagem Matemática

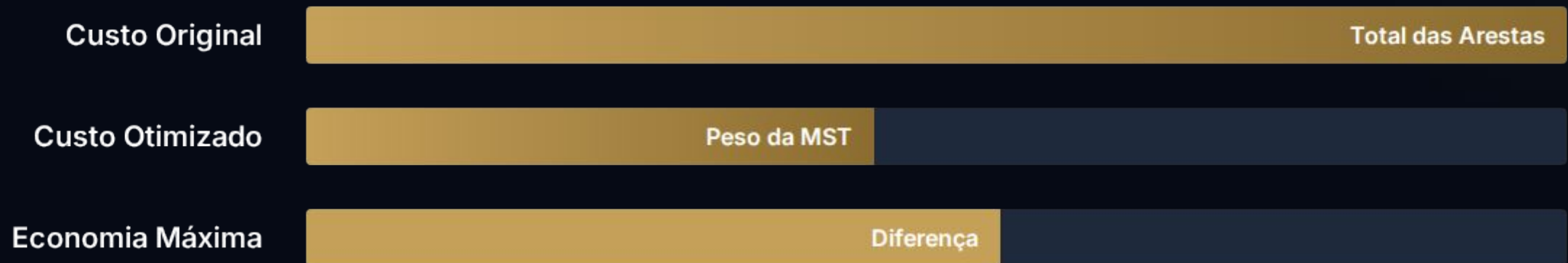
Transformando a geografia em estrutura de dados.

| Tradução para Grafo

- ✓ **Vértices:** Representam as junções (m).
- ✓ **Arestas:** Representam as estradas (n).
- ✓ **Pesos:** Comprimento em metros (z).

O problema se torna a busca por uma **Árvore Geradora Mínima (MST)** em um grafo ponderado e não-direcionado.

Comparativo de Custos



$$\text{Economia} = \sum \text{Custo}_{\text{Total}} - \sum \text{Custo}_{\text{MST}}$$

Estratégia Algorítmica

A lógica por trás da economia máxima.

| Por que o Algoritmo de Kruskal?



Abordagem Gulosa

Seleciona sempre a aresta de menor custo disponível que não forma ciclos.



Escalabilidade

Excelente performance para grafos com muitas arestas (n até 200.000).



Estruturas Fortes

Fácil integração com a estrutura Disjoint Set Union (DSU) para detecção de ciclos.

| Complexidade e Eficiência

$O(E \log E)$

Complexidade de Tempo

Análise Técnica

A ordenação das arestas domina o tempo de execução. O uso de Union-Find com compressão de caminho torna as operações de união quase constantes.

Complexidade de Espaço: $O(V + E)$ para armazenar o grafo e as estruturas de suporte.

| Considerações de Implementação

 Java code structure on a screen

- ✓ **Overflow:** Uso de long para acumular custos que podem exceder 231.
- ✓ **Multi-test cases:** Loop de execução contínua até encontrar $m=n=0$.
- ✓ **Conectividade:** Garantia de que o grafo de entrada é sempre conexo.